

Lecture 7: Gradient descent methods

JIANG, Rujun

School of Data Science
Fudan University

Iterative algorithm

Consider an unconstrained optimization problem with differentiable f :

$$\min_{x \in \mathbb{R}^n} f(x)$$

iterative algorithm framework: choose an initial point x_0 , and repeat

$$x_{k+1} = x_k + t_k p_k, \quad k = 0, 1, 2, \dots$$

If $p_k^T \nabla f(x_k) < 0$, then p_k is a descent direction.

Intuition: descending a mountain in the fog

The “Foggy Mountain” Analogy: Imagine you are trapped on a mountain in dense fog. Your goal is to reach the bottom of the valley (the minimum, x^*).

Because of the fog, you cannot see the global terrain. You can only feel the slope of the ground **directly beneath your feet**.

How do you decide your next step?

- ▶ **Direction (p_k):** You find the direction where the ground slopes downwards the steepest. This local steepest descent direction is exactly the negative gradient $-\nabla f(x_k)$.
- ▶ **Step size (t_k):** You take a step in that direction. The length of your step depends on how confident you are or how steep the slope is.
- ▶ **Iteration:** You repeat this process, evaluating the new slope at each new position, until you reach a flat area (the bottom, $\nabla f(x_k) \approx 0$).

Gradient descent method

In classical gradient descent method, $p_k = -\nabla f(x_k)$.

The direction $-\nabla f(x_k)$ is also known as the steepest descent direction.

Interpretation: minimize the linearization of f at x_k and a proximal term that keeps x_{k+1} close to x_k :

$$x_{k+1} = \operatorname{argmin}_x f(x_k) + \nabla f(x_k)^T(x - x_k) + \frac{1}{2t_k} \|x - x_k\|^2$$

Algorithm 1 Gradient descent method

```
1: initial point  $x_0$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   choose appropriate step size  $t_k$ 
4:   update  $x_{k+1} = x_k - t_k \nabla f(x_k)$ 
5:   if termination criteria is met then
6:     stop
7:   end if
8: end for
```

Termination criteria

the first order condition $\|\nabla f(x)\| = 0$ is not practical in finite precision arithmetic

practical termination criteria:

- ▶ gradient is small: $\|\nabla f(x_k)\| \leq \epsilon$
- ▶ successive objective condition $|f(x_{k+1}) - f(x_k)| \leq \epsilon$ or the relative one

$$\frac{|f(x_{k+1}) - f(x_k)|}{|f(x_k)|} \leq \epsilon$$

- ▶ successive point difference $\|x_{k+1} - x_k\| \leq \epsilon$ or the relative one

$$\frac{\|x_{k+1} - x_k\|}{\|x_k\|} \leq \epsilon$$

- ▶ to avoid division by tiny numbers (unstable division), we can replace the denominators by $\max\{1, |f(x_k)|\}$ and $\max\{1, \|x_k\|\}$, respectively

Step size rule

step size rule

- ▶ constant step size

$$t_k = \alpha \text{ for some constant } \alpha > 0$$

- ▶ exact line search

$$t_k = \operatorname{argmin}_{t \geq 0} f(x_k + tp_k)$$

- ▶ backtracking line search (most practical)

Advantages of gradient method

- ▶ every iteration is inexpensive
- ▶ does not require second order derivatives

Disadvantage

- ▶ slow convergence for ill-conditioned problems

Exact line search

Compute the exact line search step size for $x^T A x + 2b^T x + c$ with $A \succ 0$

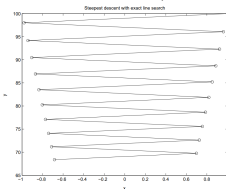
Example

$$f(x) = \frac{1}{2}((x^{(1)})^2 + \gamma(x^{(2)})^2) \text{ (with } \gamma \geq 1)$$

with exact line search and starting point $x_0 = (\gamma, 1)$

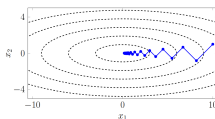
$$\frac{\|x_k - x^*\|_2}{\|x_0 - x^*\|_2} = \left(\frac{\gamma - 1}{\gamma + 1} \right)^k$$

$f(x, y) = 100x^2 + y^2$ with initial point (1, 100)



gradient method is often slow; convergence very dependent on scaling
the gradient method behaves in "zig-zag" motion: In fact, for **exact line search** gradient descent method, we have

$$(x_{k+2} - x_{k+1})^T (x_{k+1} - x_k) = 0.$$



Armijo rule line search

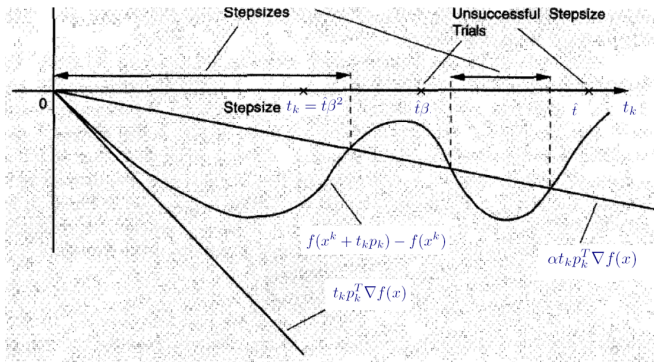
backtracking line search:

initialize t_k at $\hat{t} > 0$ (for example, $\hat{t} = 1$)

find out the smallest nonnegative integer s such that

$$f(x_k + t_k p_k) \leq f(x_k) + \alpha t_k p_k^T \nabla f(x_k),$$

with $t_k = \hat{t}\beta^s$, $0 < \beta < 1$, e.g., $\hat{t} = 1$, $\beta = 1/2$, and α is a small positive number, e.g., 10^{-4} .



Convergence analysis for gradient method

Analyze the convergence for gradient descent method

$$x_{k+1} = x_k - t_k \nabla f(x_k)$$

Assume

- ▶ f is differentiable with $\text{dom}(f) = \mathbb{R}^n$
- ▶ $\nabla f(x)$ is Lipschitz continuous with parameter $L > 0$
- ▶ optimal value is bounded

Lipschitz continuity

Lipschitz continuity: a function h is Lipschitz continuous with Lipschitz constant L if

$$\|h(x) - h(y)\|_2 \leq L \|x - y\|_2$$

for all $x, y \in \text{dom}h$.

The class of functions with Lipschitz gradient with constant L is denoted by $C_L^{1,1}(\mathbb{R}^n)$ or just $C_L^{1,1}$. Occasionally, when the exact value of the Lipschitz constant is unimportant, we will omit it and denote the class by $C^{1,1}$.

Examples: $a^T x + b$, $x^T A x + 2b^T x + c$

Properties of Lipschitz continuity

If ∇f is Lipschitz continuous with parameter L (or called L -smooth), then (**quadratic upper bound**):

$$f(y) \leq f(x) + \nabla f(x)^T (y - x) + \frac{L}{2} \|x - y\|_2^2,$$

which is equivalent to

$$\frac{Lx^T x}{2} - f(x) \text{ is convex}$$

(i.e., if f is twice differentiable, $\nabla^2 f \preceq LI$ for all x)

Let f be a twice continuously differentiable function over $\mathbb{R}^n$. Then $f \in C_L^{1,1}(\mathbb{R}^n)$ if and only if $\|\nabla^2 f(x)\| \leq L$ for any $x \in \mathbb{R}^n$.

Example: $f(x) = \sqrt{1 + x^2}$

Convergence analysis: general function

suppose that

- ▶ ∇f is Lipschitz continuous with parameter L
- ▶ f is bounded below over $\mathbb{R}^n$, that is, there exists m such that $f(x) > m$ for all $x \in \mathbb{R}^n$

then we have the following:

- ▶ **Sufficient decrease lemma:** For any $x \in \mathbb{R}^n$ and $t > 0$:

$$f(x) - f(x - t\nabla f(x)) \geq t(1 - \frac{Lt}{2})\|\nabla f(x)\|^2.$$

- ▶ constant step size: $f(x) - f(x - t\nabla f(x)) \geq \bar{t}(1 - \frac{L\bar{t}}{2})\|\nabla f(x)\|^2$
a popular choice: $\bar{t} = 1/L$
- ▶ exact line search: $f(x) - f(x - \bar{t}\nabla f(x)) \geq \frac{1}{2L}\|\nabla f(x)\|^2$
- ▶ backtracking: $f(x) - f(x - t\nabla f(x)) \geq \alpha \min\{\hat{t}, \frac{2(1-\alpha)\beta}{L}\}\|\nabla f(x)\|^2$

- **Convergence of the gradient method:** The sequence $\{f(x_k)\}_{k \geq 0}$ is nonincreasing. In addition, for any $k \geq 0$, $f(x_{k+1}) < f(x_k)$ unless $\nabla f(x_k) = 0$. $\nabla f(x_k) \rightarrow 0$ as $k \rightarrow \infty$
- **Rate of convergence of gradient norms:** Let f^* be the limit of the convergent sequence $\{f(x_i)\}_{i \geq 0}$. Then for any $n = 0, 1, 2, \dots$

$$\min_{i=0,1,\dots,k} \|\nabla f(x_i)\| \leq \sqrt{\frac{(f(x_0) - f^*)}{(k+1)M}},$$

where $M = t(1 - \frac{Lt}{2})$.

- **Remark:** This $O(1/\sqrt{k})$ rate ensures convergence to a stationary point (e.g., a local minimum), but does not guarantee finding the global optimum.

For exact line search and backtracking line search methods, we have similar results but with slightly different constants.

Convergence analysis of constant step size: convex function

If the step size is a constant t ,

$$x_{k+1} = x_k - t \nabla f(x_k)$$

$$f(x_k) - f(x^*) \leq \frac{1}{2kt} \|x_0 - x^*\|_2^2$$

Hence, the iterate number for $f(x_k) - f(x^*) \leq \epsilon$ is $O(1/\epsilon)$.

We also have $\|\nabla f(x_k)\|^2 \leq 2L(f(x_k) - f^*) \leq \frac{L}{kt} \|x_0 - x^*\|_2^2$.

Can be extended to case of line search if we assume $\alpha = 1/2$

There exist fast gradient methods proposed by Nesterov, which has a convergence rate $O(1/k^2)$.

Strongly convex case

μ -strongly convex: f is strongly convex with parameter $\mu > 0$ if

$$f(x) - \frac{\mu}{2}x^Tx \text{ is convex}$$

First order condition (**quadratic lower bound**)

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) + \frac{\mu}{2} \|x - y\|_2^2$$

Second order condition

$$\nabla^2 f(x) \succeq \mu I$$

linear convergence $f(x_k) - f^* \leq (1 - \frac{\mu}{L})^k (f(x_0) - f^*)$

Can be extended to case of line search if we assume $\alpha = 1/2$

Condition number

generalization of condition number of a p.d. matrix A : $\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$

number of iterations to reach $f(x_k) - f^* \leq \epsilon$ is at most

$$\frac{\log((f(x_0) - f^*)/\epsilon)}{\log((1 - \mu/L)^{-1})} \approx \frac{L}{\mu} \log\left(\frac{f(x_0) - f^*}{\epsilon}\right)$$

L/μ is known as the condition number

from the convergence rate $(1 - \frac{\mu}{L})^k$, we can find that for constant step size gradient method, the main factor that influences the speed is the condition number

for the example in page 6, $f(x) = \frac{1}{2}((x^{(1)})^2 + \gamma(x^{(2)})^2)$: when $\gamma = 10000$ and $k = 100$, we have $\left(\frac{\gamma-1}{\gamma+1}\right)^k = 0.98$

gradient method can be very slow, and very much dependent on scaling

Condition number: geometric intuition

Condition number $\kappa = L/\mu$ determines the **shape** of the function's contours:

Well-conditioned ($\kappa \approx 1$)

- ▶ **Shape:** A “round bowl”.
- ▶ **Contours:** Nearly circular.
- ▶ **Behavior:** Gradient points directly toward the center.
- ▶ **Convergence:** Extremely fast.

Ill-conditioned ($\kappa \gg 1$)

- ▶ **Shape:** A “narrow valley”.
- ▶ **Contours:** Highly eccentric ellipses.
- ▶ **Behavior:** Gradient is nearly perpendicular to the long axis.
- ▶ **Convergence:** Slow “zig-zag” motion.

Question: Why does this geometric shape dictate the speed?

Why High κ Slows Down Gradient Descent

1. Curvature Disparity:

In a “narrow valley”, the curvature is extremely unbalanced:

- ▶ Cross-valley (L): Steep walls (large eigenvalue).
- ▶ Along-valley (μ): Flat floor (small eigenvalue).

2. The “Global Step Size” Flaw: Standard GD uses a single t for all directions. It cannot take “small steps on walls” and “giant steps on the floor” simultaneously.

3. The Core Conflict: Stability vs. Progress

- ▶ The Wall’s Warning (Stability): To avoid “flying out” of the valley (divergence), we must set $t < 2/L$ (typically, we set $t = 1/L$ for optimal descent).
- ▶ The Floor’s Frustration (Progress): In the direction of the optimum, the gradient is already weak ($\approx \mu$). Multiplying this by a tiny t results in **negligible progress**.

Result: $O(L/\mu)$ iterations are needed to escape the valley.

Preconditioned Gradient Descent

Motivation: If the slow convergence is caused by the “narrow valley” (high condition number κ), can we reshape the terrain?

Preconditioning: Apply a linear transformation to the space to make the condition number $\kappa \approx 1$ (i.e., stretch the valley into a round bowl).

Algorithm Update: Instead of the steepest descent direction $p_k = -\nabla f(x_k)$, we use a symmetric positive definite matrix $P \succ 0$:

$$x_{k+1} = x_k - t_k P \nabla f(x_k)$$

Common Choices of P :

- **Diagonal Matrix (Jacobi Preconditioner):** $P = \text{diag}(d_1, \dots, d_n)$.

Rule of thumb: $d_i = \left(\frac{\partial^2 f}{\partial x_i^2} \right)^{-1}$.

Intuition: Inverse curvature. Assign a small step for steep walls (large 2nd derivative), and a large step for flat floor (small 2nd derivative).

- **Inverse Hessian:** $P = (\nabla^2 f(x_k))^{-1}$.

Preview: This perfectly corrects the local curvature in all directions, leading to **Newton's Method!**

Convergence rate

- ▶ Problem error measurements:

- ▶ $e_k = \|x_k - x^*\|$

- ▶ $e_k = f(x_k) - f(x^*)$

- ▶ $e_k = \|\nabla f(x_k)\|$

- ▶ Convergence

$$\lim_{k \rightarrow \infty} x_k = x^* \text{ or } \lim_{k \rightarrow \infty} f(x_k) = f(x^*)$$

- ▶ Iteration complexity:

$$e_k \leq O\left(\frac{1}{k}\right) \left(O\left(\frac{1}{k^2}\right), O\left(\frac{1}{\sqrt{k}}\right), O\left(\frac{1}{2^k}\right), O\left(\frac{1}{2^{2^k}}\right) \right)$$

Convergence rate

Suppose that sequence $\{x_k\}$ converges to x^* . For $e_k = \|x_k - x^*\|$ or $e_k = f(x_k) - f(x^*)$, consider $\lim_{k \rightarrow \infty} \frac{e_{k+1}}{e_k} = \mu$

- ▶ if $\mu = 1$, then $\{x_k\}$ converges (Q-)sublinearly¹
- ▶ if $\mu = (0, 1)$, then $\{x_k\}$ converges (Q-)linearly
- ▶ if $\mu = 0$, then $\{x_k\}$ converges (Q-)superlinearly
- ▶ To distinguish superlinear rates of convergence, we check

$$\lim_{k \rightarrow \infty} \frac{e_{k+1}}{(e_k)^q} = \mu > 0$$

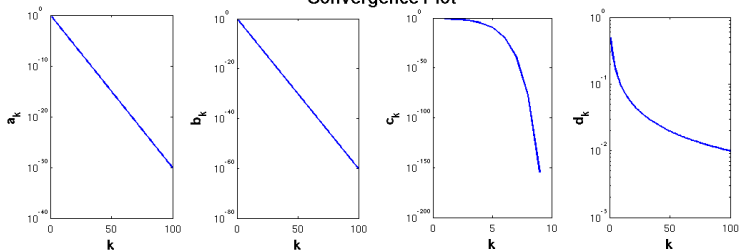
if $q = 2$, the sequence is quadratic convergence

¹The "Q-" stands for "quotient" because the terms are defined using the quotient between two successive terms. Extension: Suppose that $\{x_k\}$ converges to x^* . The sequence is said to converge R-linearly to x^* if there exists a sequence $\{\varepsilon_k\}$ such that $\|x_k - x^*\| \leq \varepsilon_k$ for all k , and (ε_k) converges Q-linearly to zero. The "R-" prefix stands for "root".

Examples

$$a_k = 1/2^k, \quad b_k = 1/4^k, \quad c_k = 1/2^{2^k}, \quad d_k = 1/(k+1)$$

Convergence Plot



Sublinear vs Linear Convergence

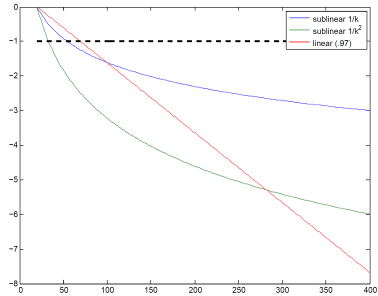
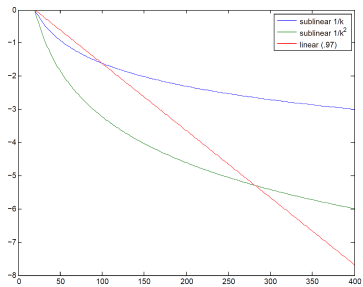
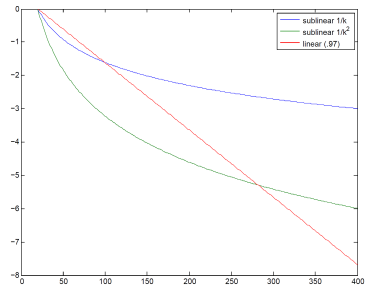
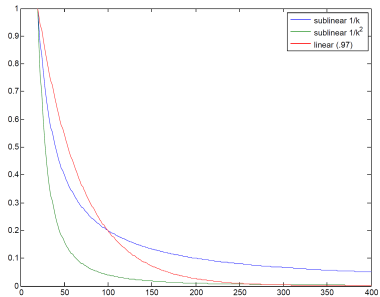
Sublinear convergence (e.g. error decreases like C/j or C/j^2 in iteration number j , for some constant C) is slow.

To achieve a specified accuracy ϵ , need $O(C/\epsilon)$ iterations for $1/k$ process, and $\sqrt{C/\epsilon}$ for a $1/k^2$ process.

Linear convergence is asymptotically faster. If error decreases like $(1 - \delta)^k$ for some small positive δ , need $O(\log(1/\epsilon)/\delta)$ iterates to reach accuracy ϵ .

Sublinear rates were often viewed almost as uninteresting. But some modern applications need only a low-accuracy solution, and a sublinear algorithm may suffice.

(The constants in the $O(\)$ terms and the cost per iteration are significant.)



Implementation

a flop (floating-point operation): one addition, subtraction, multiplication or division of two floating-point numbers

to evaluate the complexity of an algorithm, we count the total number of flops, express it as a function of the dimensions of the matrices and vectors involved, and simplify the expression by ignoring all terms except the leading terms.

example: $m^3 + 3m^2n + mn + 4mn^2 + 5m + 22$

line searches: $f(x + td)$ can sometimes be evaluated by first forming $z = x + td$ and then $f(z)$

line searches with affine composition: $\phi(Ax + b)$

quadratic function: $f(x) = \frac{1}{2}x^T Ax + b^T x$

Example: logistic regression

$$\min \ell(x) := \frac{1}{m} \sum_{i=1}^m \log(1 + \exp(-b_i a_i^T x)) + \lambda \|x\|^2,$$

where the label $b_i \in \{-1, 1\}$

- Let $p_i(x) = \frac{1}{1 + \exp(-b_i a_i^T x)}$, $p(x) = (p_1(x), \dots, p_m(x))^T$ and $A = [a_1, \dots, a_m]^T \in \mathbb{R}^{m \times n}$. Then $b_i a_i^T x = [b \circ (Ax)]_i$.

MATLAB command `p=1./(1+exp(-b.*(A*x)))`

Computing $p(x)$ takes $O(mn)$ flops (or $O(N)$ flops if A has N nonzero entries)

- function value: $\ell(x) = -\frac{1}{m} e^T \log(p(x)) + \lambda \|x\|^2$
- Gradient: $\nabla \ell(x) = -\frac{1}{m} A^T (b \circ (e - p(x))) + 2\lambda x$